

From Extension Methods to Extension Everything: C# 14's Bold Evolution in Type Enhancement

*Why the next version of C# might
change how you architect .NET systems
forever*



Balagurunathan Marimuthu
@gmbalaa

Swipe →

The Relatable Problem

You Know This Pain...

You need to add functionality to a class
you CAN'T modify:

- ✗ Vendor library
- ✗ Legacy code
- ✗ Core domain model

But your team needs:

- Analytics properties
- Reporting formats
- Business calculations

What do you do?



Balagurunathan Marimuthu
@gmbalaa

Swipe →

Current Solution #1

Option A: Wrapper Classes

Create:

- AnalyticsOrder
- ReportingOrder
- PricingOrder

Result:

- 😞 Mapping code everywhere
- 😞 3 versions of "same thing"
- 😞 Endless boilerplate

We've all done this. It never feels right.



Balagurunathan Marimuthu
@gmbalaa

Swipe →

Current Solution #2

Option B: Extension Methods

Better, but...

- ✓ Less boilerplate
- ✗ Only methods work
- ✗ No properties
- ✗ No indexers
- ✗ Static classes everywhere

Still feels like a workaround.



Balagurunathan Marimuthu
@gmbalaa

Swipe →

Current Solution #3

Option C: Just Add Everything

Jam it all into the base class:

```
class Order
{
    // Domain stuff
    // Analytics stuff
    // Reporting stuff
    // Pricing stuff
    // ...
}
```



Single Responsibility? Never heard of her. 😅



Balagurunathan Marimuthu
@gmbalaa

Swipe →

The Reveal

C# 14's Extension Types

What if you could extend:

- ✓ Properties
- ✓ Methods
- ✓ Indexers
- ✓ Operators
- ✓ ..even Generic Types



All without touching the original class?

Let me show you each one →



Balagurunathan Marimuthu
@gmbalaa

Swipe →

Base Class (Reference Point)

```
Order.cs

1  // The Order class we can't MODIFY
2  // from Vendor library or legacy code
3  public class Order
4  {
5      public int Id { get; set; }
6      public decimal TotalAmount { get; set; }
7      public DateTime CreatedDate { get; set; }
8      public List<OrderItem> Items { get; set; } = [];
9  }
10
11 public class OrderItem
12 {
13     public int ProductId { get; set; }
14     public string ProductName { get; set; } = string.Empty;
15     public int Quantity { get; set; }
16     public decimal UnitPrice { get; set; }
17
18     public bool IsValid =>
19         ProductId > 0 &&
20         !string.IsNullOrEmpty(ProductName) &&
21         Quantity > 0 &&
22         UnitPrice > 0;
23 }
24
25 // That's it. Clean. Focused.
26 // Now, let's extend it...
```



Balagurunathan Marimuthu
@gmbalaa

Swipe →

Properties Extension

```
OrderExtensions.cs

1 public static partial class OrderExtensions
2 {
3     // Properties and categorization logic for analytics
4     extension(Order order)
5     {
6         public decimal AverageItemValue =>
7             order.Items.Count > 0
8                 ? order.TotalAmount / order.Items.Count
9                 : 0;
10
11         public int DaysSinceOrder =>
12             (DateTime.Now - order.CreatedDate).Days;
13
14         public string RevenueCategory =>
15             order.TotalAmount switch
16             {
17                 < 100 => "Low Value",
18                 < 500 => "Medium Value",
19                 _ => "High Value"
20             };
21
22         public bool IsAtRisk =>
23             order.DaysSinceOrder > 30 && order.TotalAmount > 500;
24     }
25 }
```



Balagurunathan Marimuthu
@gmbalaa

Swipe →

Methods Extension

```
OrderExtensions.cs

1 public static partial class OrderExtensions
2 {
3     // Methods extending Order for business operations
4     extension(Order order)
5     {
6         public decimal CalculateDiscount() =>
7             order.TotalAmount switch
8             {
9                 >= 1000 => order.TotalAmount * 0.15m,
10                >= 500 => order.TotalAmount * 0.10m,
11                >= 100 => order.TotalAmount * 0.05m,
12                _ => 0
13            };
14
15         public bool ValidateForCheckout() =>
16             order.Items is { Count: > 0 } &&
17             order.TotalAmount > 0 &&
18             order.Items.All(i => i.IsValid);
19     }
20 }
```



Balagurunathan Marimuthu
@gmbalaa

Swipe →

Operators Extension

```
OrderExtensions.cs

1 public static partial class OrderExtensions
2 {
3     // Overloaded comparison operators for Order
4     extension(Order order)
5     {
6         public static bool operator >(Order left, Order right) =>
7             left.TotalAmount > right.TotalAmount;
8
9         public static bool operator <(Order left, Order right) =>
10             left.TotalAmount < right.TotalAmount;
11
12         public static bool operator >=(Order left, Order right) =>
13             left.TotalAmount >= right.TotalAmount;
14
15         public static bool operator <=(Order left, Order right) =>
16             left.TotalAmount <= right.TotalAmount;
17
18         public static bool operator ==(Order left, Order right) =>
19             left.TotalAmount == right.TotalAmount;
20
21         public static bool operator !=(Order left, Order right) =>
22             left.TotalAmount != right.TotalAmount;
23     }
24 }
```



Balagurunathan Marimuthu
@gmbalaa

Swipe →

Indexers Extension

Not yet fully supported; below is the workaround

```
OrderExtensions.cs

1 public static partial class OrderExtensions
2 {
3     // Indexer-like accessor for dynamic property retrieval
4     extension(Order order)
5     {
6         public string Get(string field) =>
7             field.ToLower() switch
8             {
9                 "id" => $"#{order.Id}",
10                "revenue" => $"${order.TotalAmount:N2}",
11                "items" => $"{order.Items.Count} items",
12                "date" => order.CreatedDate.ToString("MMM dd, yyyy"),
13                "status" => order.TotalAmount > 500
14                    ? "High Value"
15                    : "Standard",
16                _ => "N/A"
17            };
18     }
19 }
```



Balagurunathan Marimuthu
@gmbalaa

Swipe →

But Wait... There's More

GENERIC EXTENSIONS

Extension types work with generics too!

Extend:

- ✓ `List<T>`
- ✓ `IEnumerable<T>`
- ✓ `Dictionary<TKey, TValue>`
- ✓ Your own generic types

Add behavior to ENTIRE families of types at once.

One extension. Infinite possibilities.

This is where it gets really powerful. 🚀



Balagurunathan Marimuthu
@gmbalaa

Swipe →

Real-world usage (All Together)

```
AnalyticsService.cs

1 // All extensions are working together
2 // Analytics service to analyze the orders.
3 public class AnalyticsService
4 {
5     public void AnalyzeOrder(List<Order> orders)
6     {
7         var insights = orders
8             .Where(o => o.DaysSinceOrder > 30)
9             .Select(o => new
10             {
11                 OrderId = o.Get("id"),           // Indexer-like accessor
12                 Category = o.RevenueCategory,     // Property
13                 Discount = o.CalculateDiscount(), // Method
14                 Average = o.AverageItemValue      // Property
15             })
16             .OrderByDescending(x => x.Discount); // operator overloads
17
18         // Clean. Natural. Type-safe.
19     }
20 }
```



Balagurunathan Marimuthu
@gmbalaa

Swipe →

Comparison

```
OrderExtensions.cs

1 public static class OrderExtensions
2 {
3     public static decimal GetAverage(this Order order)
4     {
5         return order.Items.Count > 0
6             ? order.TotalAmount / order.Items.Count
7             : 0;
8     }
9 }
10
11 // Usage Example:
12 // var average = order.GetAverage(); → Method call
```

```
OrderExtensions.cs

1 public static class OrderExtensions
2 {
3     extension(Order order)
4     {
5         public decimal AverageItemValue ⇒
6             order.Items.Count > 0
7                 ? order.TotalAmount / order.Items.Count
8                 : 0;
9     }
10 }
11
12 // Usage Example (natural):
13 // var average = order.AverageItemValue; → Property access
```



Balagurunathan Marimuthu
@gmbalaa

Swipe →

The Payoff

What You Just Saw:

- ✓ Properties (not just methods)
- ✓ Methods (when you need them)
- ✓ Operators (natural comparison)
- ✓ Generic extensions (scale it up)
- ✓ Indexers (not fully supported...)

All working on ONE class that you never modified.

No wrappers.

No mapping.

No compromise.

This is composition at the language level.



Balagurunathan Marimuthu
@gmbalaa

Swipe →

Real Generic Use cases

Generic Extensions in Action

Imagine Extending:

 Collections - Add statistical methods to all `List<T>` types at once

 LINQ Results - Enhance `IEnumerable<T>` with domain-specific filters

 Repositories - Add audit trails to all `IRepository<T>` implementations

 API Responses - Add serialization helpers to `Result<T>` types

One extension type = Works everywhere.



Balagurunathan Marimuthu
@gmbalaa

Swipe 

The Honest Take

Let's Be Real:

Concerns ⚠️

- How do teams discover extensions?
- What about naming conflicts?
- Migration from old code?
- Generic constraint complexity?

But Compare To Now:

- Wrapper classes everywhere
- Mapping code bloat
- Extension method hacks
- God classes doing everything
- Duplicated logic across types

After a decade of workarounds... This solves real problems.



Balagurunathan Marimuthu
@gmbalaa

Swipe →

If you
find this
helpful, please
like and share it
with your friends

Follow
for more **.NET &**
C# tips



Balagurunathan Marimuthu
@gmbalaa